

■ INVENTARIO TÉCNICO EXHAUSTIVO

ShoriExpress - Sistema de Gestión de Restaurante

Generado: 20 de April de 2026 a las 07:20

CONTENIDO	
1. Arquitectura General	4
2. Desglose de Módulos	5
- App: Rol	5
- App: Usuario	6
- App: Inventario	8
- App: Movimiento Inventario	10
- App: Producto	11
- App: Receta	13
- App: Pedido	14
- App: Detalle Pedido	16
- App: Método Pago	16
- App: Recibo	17
- App: Dashboard	18
- App: Cuentas	19
3. Funcionalidades Transversales	20
4. Interfaz de Usuario	21
5. Validaciones y Reglas de Negocio	22
6. 85+ Historias de Usuario	23

■ 1. ARQUITECTURA GENERAL

Stack Tecnológico

- Framework: Django 4.2.29
- Base de Datos: MySQL (primaria) / SQLite (desarrollo)
- Puerto: 3307 (MySQL)
- Timezone: America/Bogota
- Charset: UTF-8MB4 (soporte caracteres especiales)
- Arquitectura: MVC con Signals para automatización

Aplicaciones Instaladas (12 apps)

1. rol - Gestión de niveles de acceso
2. usuario - Gestión de identidades (clientes y empleados)
3. inventario - Control de stock e insumos
4. producto - Catálogo de productos/platos
5. receta - Escandallo (ingredientes requeridos)
6. movimiento_inventario - Auditoría de cambios
7. pedido - Órdenes de compra
8. detalle_pedido - Items por pedido
9. metodo_pago - Formas de pago
10. recibo - Facturas y contabilidad
11. dashboard - Analytics y KPIs
12. cuentas - Autenticación y landing

Decoradores de Autenticación

- @login_shori_required - Usuarios logueados (clientes y empleados)
- @admin_shori_required - Solo roles admin/administrador/empleador

■ 2. DESGLOSE DE MÓDULOS

APP: ROL

Gestión de niveles de acceso del sistema. Modelo simple con campo 'nombre_rol' único. Endpoints: CRUD completo (crear, leer, actualizar, eliminar). Validación: nombres únicos.

APP: USUARIO

Gestión de identidades para clientes y empleados. Campos: tipo_documento, documento (único), nombre_usuario (único, 4-50 chars), contraseña (hash PBKDF2 + soporte legado), email (único), teléfono (10 dígitos), dirección, rol (FK), estado (activo/inactivo), bonos_fidelidad (0-10).

Validaciones: regex para username/documento/nombre/teléfono, contraseña ≥8 caracteres en registro. Endpoints: CRUD usuario + login + register + mi_cuenta + cambio de contraseña (solo empleados).

APP: INVENTARIO

Control de insumos con modelo de lotes. Campos: nombre_insumo, categoría, unidad_medida (GR/KG/ML/LT/UN), stock_actual (≥0), stock_minimo, stock_maximo, precio_compra, iva (0-100%), estado_insumo (disponible/pocos/agotado).

Modelo InventarioLote: Tracks stock por lote con código y fecha_vencimiento. Validaciones: stock_minimo ≤ stock_maximo, no negativos, estado calculado automáticamente. Lock pessimista (select_for_update) para evitar race conditions.

APP: MOVIMIENTO_INVENTARIO

Auditoría completa de cambios de stock. Tipos: entrada, entrada_inicial, salida_venta, salida_desperdicio, ajuste. Campos: insumo, usuario, cantidad (>0), lote, fecha_vencimiento, observaciones. Validaciones: salidas no pueden exceder stock disponible.

APP: PRODUCTO

Catálogo de platos/productos. Campos: nombre_producto, descripción, imagen (ImageField), imagen_catalogo (ruta estática), precio_venta, es_combo (boolean), esta_disponible. Sistema de carrito en sesión Django con funciones add/remove/decrement/clear. Validación de stock máximo antes de agregar al carrito.

APP: RECETA

Escandallo: relación producto-insumo con cantidad_requerida. Validación unique_together (producto, insumo). Se usa para calcular máximo de unidades disponibles en carrito.

APP: PEDIDO ■ CRÍTICO

Órdenes de compra con máquina de estados: pendiente → preparacion → listo → entregado (o cancelado). Campos: usuario, tipo_pedido (local/llevar/domicilio), dirección, estado, total, fecha_pedido, fecha_entrega_estimada (ahora + 45 min), fecha_entrega_real.

Signal 1: pre_save captura estado anterior y fija fecha_entrega_real en transición a entregado. Signal 2: post_save dispara procesar_inventario_por_pedido() al entrar en 'preparacion': - Para CADA ingrediente: valida stock, crea MovimientoInventario, decrementa stock, recalcula estado. - Transacción atómica con lock pessimista.

APP: DETALLE_PEDIDO

Items del pedido. Campos: pedido (FK), producto (FK), cantidad, precio_unitario_momento (auditoría de precio), notas_especiales (ej: "Sin cebolla"). Propiedad: subtotal = cantidad * precio.

APP: METODO_PAGO

Formas de pago (Efectivo, Tarjeta, Transferencia, etc.). Campos: nombre_metodo (único), descripción, esta_activo (boolean). Siempre filtrado por esta_activo=True.

APP: RECIBO

Facturas contables. Relación 1:1 con Pedido. Campos: pedido, metodo_pago, fecha_emision, subtotal, iva_total, total_pagado, puntos_ganados (0 o 1). Lógica: Si total ≥ \$50,000 → puntos_ganados=1 → usuario.bonos_fidelidad += 1 (máx 10).

APP: DASHBOARD

Analytics y KPIs. Endpoint API: /dashboard/api/dashboard-data/ retorna JSON con: - KPIs: total_ventas, total_pedidos, pedidos_hoy, pedidos_pendientes, recibos_completados, usuarios_activos, insumos_bajo_stock, total_productos. - Gráficos: pedidos_por_estado, ventas_por_día, productos_más_vendidos, métodos_pago_frecuencia. Filtrable por fecha_inicio y fecha_fin (default: últimos 30 días).

APP: CUENTAS

Landing page y autenticación. Endpoints: landing (portada), login/register (auto-registro clientes), logout, ver_carrito (con bonos), menu_publico, api_hora_bogota, mis_pedidos, detalle_pedido. Seguridad: url_has_allowed_host_and_scheme para prevenir open-redirect, CSRF protection. API hora: Intenta worldtimeapi.org, fallback a timezone.now() de Django.

■ 3. LÓGICA DE NEGOCIO CRÍTICA

Sistema de Bonos de Fidelidad

- Umbral: Si $\text{total_pedido} \geq \$50,000 \rightarrow$ Usuario gana 1 bono
- Máximo: Usuario acumula máx 10 bonos
- Redención: 5 bonos = 5% descuento en siguiente compra
- Validación: $\text{bonos_fidelidad} \geq 5$ y $\text{total_pedido} > 0$ para redimir

Flujo de Checkout Crítico (finalizar_compra)

1. Validar usuario logueado, carrito no vacío, método Efectivo activo
 2. Iterar carrito: validar cantidad (1-500), validar precio no cambió
 3. Si redención: $\text{total} *= 0.95$, $\text{usuario.bonos} -= 5$
 4. Crear Pedido (estado='pendiente'), DetallePedidos, Recibo
 5. Si $\text{total} \geq \$50,000$: $\text{usuario.bonos} += 1$ (máx 10)
 6. Limpiar carrito
- Transacción atómica. Si error $\rightarrow$ ROLLBACK.

Descuento de Inventario (Signal Crítico)

SE DISPARA cuando: Pedido cambia a estado='preparacion'

Para CADA detalle $\rightarrow$ Para CADA ingrediente (receta):

- $\text{cantidad_a_descontar} = \text{cantidad_requerida} \times \text{detalle.cantidad}$
 - Validar $\text{stock_actual} \geq \text{cantidad_a_descontar}$ (sino $\rightarrow$ EXCEPTION)
 - Crear MovimientoInventario tipo='salida_venta'
 - Decrementar $\text{Inventario.stock_actual}$
 - Recalcular estado: agotado (≤ 0), pocos ($\leq \text{mínimo}$), disponible
- TODO en transacción atómica con lock pessimista.

Validación de Máximo Disponible (Carrito)

$\text{max_unidades} = \text{MIN}(\text{stock_insumo}_1 / \text{cantidad_req}_1, \dots, \text{stock_insumo}_n / \text{cantidad_req}_n)$

Si $(\text{cantidad_actual} + 1) > \text{max} \rightarrow$ No permitir agregar.

Si $\text{max} = 0 \rightarrow$ Mostrar "Sin stock"

Semáforo de Entregas

$\text{diff_min} = (\text{referencia} - \text{fecha_estimada}).\text{total_seconds}() / 60$

referencia = fecha_entrega_real (si entregado) o ahora

- Cancelado: NEUTRAL (gris)
- Entregado + $\text{diff} \leq 10$ min: SUCCESS (verde) "A tiempo"
- Entregado + $\text{diff} > 10$ min: DANGER (rojo) "Con demora"
- Pendiente/Prep/Listo + $\text{diff} > 10$: DANGER "Tarde"
- Pendiente/Prep/Listo + $-10 \leq \text{diff} \leq 10$: WARNING (amarillo) "En riesgo"
- Pendiente/Prep/Listo + $\text{diff} < -10$: SUCCESS "A tiempo"

Contabilidad (Recibo)

$\text{subtotal} = \text{total_pedido} / 1.19$

$\text{iva_total} = \text{total_pedido} - \text{subtotal}$

Si $\text{total} \geq \$50,000$: $\text{puntos_ganados} = 1 \rightarrow \text{usuario.bonos_fidelidad} += 1$

Auditoría de Precios

$\text{DetallePedido.precio_unitario_momento} = \text{precio_al_momento_venta}$

Esto permite históricamente saber cuánto pagó el cliente si el precio maestro cambió.

■ 4. VALIDACIONES Y REGLAS DE NEGOCIO

Registro de Usuario

- Campos obligatorios: tipo_doc, documento, username, password, nombre, apellido, correo, teléfono, dirección
- Username: Regex $^[A-Za-z0-9_]{4,50}\$$
- Documento: Regex $^[A-Za-z0-9]{5,20}\$$ (sin espacios)
- Nombre/Apellido: Solo letras + tildes, 2-40 chars
- Teléfono: Exactamente 10 dígitos $^[0-9]{10}\$$
- Contraseña: Mínimo 8 caracteres
- Unicidad: username, documento, correo
- Rol default: Cliente (se asigna automáticamente)

Creación de Insumo

- Validación Decimal: $\text{cant_inicial} \geq 0$, $\text{stock_minimo} \geq 0$, $\text{stock_maximo} \geq 0$, $\text{precio} \geq 0$
- Validación relación: $\text{stock_minimo} \leq \text{stock_maximo}$
- IVA: 0-100
- Si $\text{cant_inicial} > 0$: Crea MovimientoInventario tipo='entrada_inicial' automáticamente
- Lock pessimista en todas operaciones

Movimientos de Inventario

- cantidad > 0 (obligatorio)
- Para salidas: $\text{stock_actual} \geq \text{cantidad}$ (sino → ERROR)
- Movimientos tipo 'ajuste' NO se pueden editar
- Lotes se ajustan automáticamente (get_or_create)
- Auditoría: Siempre registra usuario responsable + fecha + observaciones

Stock Negativo

- NUNCA permitido. Todas las salidas validadas previamente.
- Si $\text{stock} \leq 0$ después cálculo → Estado='agotado' automáticamente.

Contraseña (Legacy + Nueva)

- Soporte simultáneo: Texto plano (legacy) y PBKDF2 (actual)
- identify_hasher() detecta formato automáticamente
- En login: Si es legado → rehash automático a PBKDF2
- Cambio contraseña: Solo empleados (NO clientes por política)

Seguridad General

- ORM Django: Previene SQL injection automáticamente
- Open redirect: url_has_allowed_host_and_scheme() en login
- CSRF: Middleware activo, requiere token en POST
- Select for update: Lock pessimista en operaciones críticas
- Transacciones atómicas: Envuelven operaciones multi-paso

Restricciones de Negocio

- Un pedido = Un recibo (OneToOne)
- Un usuario = Múltiples pedidos (1:N)
- Un insumo = Múltiples lotes (1:N)
- Producto NO puede ser borrado si tiene detalles de pedido (PROTECT)
- Usuario NO puede ser borrado si tiene pedidos (CASCADE - se borra todo)
- Rol NO puede ser borrado si tiene usuarios (PROTECT)

■ 5. ENDPOINTS PRINCIPALES POR APP

/usuarios/ (CRUD)

GET: Listar usuarios | POST: Crear usuario | PUT/POST: Editar | POST: Eliminar

/rol/ (CRUD)

GET: Listar roles | POST: Crear | POST: Editar | POST: Eliminar

/inventario/ (CRUD + Auditoría)

GET: Listar insumos con lotes | POST: Crear insumo | POST: Editar | POST: Eliminar

GET: /inventario/reporte/pdf/ → Reporte PDF

/movimientos/ (Auditoría)

GET: Listar movimientos | POST: Crear movimiento | POST: Editar | POST: Eliminar

/productos/ (CRUD + Carrito)

GET: Listar (admin) | POST: Crear | POST: Editar | POST: Eliminar

GET: /productos/menu/ → Menú público (solo disponibles)

POST: /productos/carrito/agregar// → Agregar (con validación stock)

GET: /productos/carrito/eliminar// → Quitar

GET: /productos/carrito/restar// → Decrementar cantidad

GET: /productos/carrito/limpiar/ → Vaciar carrito

/recetas/ (CRUD)

GET: Listar recetas agrupadas | POST: Crear ingrediente | POST: Editar | POST: Eliminar

/pedidos/ (Órdenes + Estados)

GET: Listar pedidos (admin con semáforo) | POST: Crear (admin manual)

GET: /pedidos/checkout/ → Resumen precompra

POST: /pedidos/confirmar/ → Finalizar compra (crea Pedido + Recibo)

POST: /pedidos/actualizar-estado// → Cambiar estado (dispara signal)

POST: /pedidos/editar// → Editar

POST: /pedidos/eliminar// → Eliminar

GET: /pedidos/reporte/pdf/ → PDF pedidos

/recibos/ (Facturas)

GET: Listar recibos | POST: Crear recibo | POST: Editar | POST: Eliminar

GET: /recibos/factura/pdf/ → Factura individual PDF

GET: /recibos/reporte/pdf/ → Reporte recibos

/dashboard/api/dashboard-data/

GET: JSON con KPIs (filtrable por fecha_inicio, fecha_fin)

Retorna: KPIs, gráficos de ventas, productos top, métodos pago

/ (Cuentas - Landing)

GET: landing/ → Portada comercial

GET/POST: login/ → Inicio sesión

GET/POST: register/ → Auto-registro (clientes)

GET: logout/ → Cierre sesión

GET: carrito/ → Ver carrito actual

GET: menu-completo/ → Catálogo completo (público)

GET: api/hora-bogota/ → JSON hora actual (worldtimeapi.org)

GET: mis-pedidos/ → Historial pedidos (cliente logueado)

GET: pedido/detalle// → Detalles pedido individual

TOTAL: 80+ endpoints funcionales

Decoradores: @login_shori_required, @admin_shori_required

■ 6. 85+ HISTORIAS DE USUARIO DERIVABLES

Gestión de Usuarios (12)

1. Registrarse como cliente | 2. Login con validación de credenciales
3. Cambiar contraseña (empleados) | 4. Cambiar username (empleados)
5. Editar perfil personal | 6. Ver historial de compras (cliente)
7. Crear usuario (admin) | 8. Editar usuario (admin)
9. Desactivar/Activar usuario | 10. Exportar lista de usuarios
11. Validar documento único | 12. Recuperar contraseña (future)

Gestión de Roles (4)

1. Crear rol | 2. Editar rol | 3. Eliminar rol | 4. Asignar permisos CRUD por rol

Inventario (18)

1. Crear insumo con stock inicial | 2. Editar insumo (stock, umbrales, precio)
3. Eliminar insumo | 4. Ver lista de insumos con estado
5. Alertas de bajo stock | 6. Registrar entrada de proveedor
7. Registrar salida por desperdicio | 8. Registrar ajuste de stock
9. Gestionar lotes por insumo | 10. Validar fecha de vencimiento
11. Generar reporte PDF de inventario | 12. Filtrar por categoría
13. Calcular valor total del inventario | 14. Auditar historial de movimientos
15. Revertir movimiento (restricciones) | 16. Bulk import de insumos
17. Configurar stock mínimo y máximo | 18. Recibir alertas en dashboard

Productos y Recetas (20)

1. Crear producto | 2. Editar producto (nombre, precio, disponibilidad)
3. Eliminar producto | 4. Ver catálogo (admin)
5. Crear receta (asignar ingredientes) | 6. Editar receta (cambiar cantidad)
7. Eliminar receta | 8. Ver recetas por producto
9. Calcular costo COGS por producto | 10. Establecer precio venta
11. Crear combo (múltiples productos) | 12. Ver productos con stock insuficiente
13. Generar etiqueta/código de producto | 14. Marcar producto como indisponible
15. Subir imagen del producto | 16. Ver descripción (tienda)
17. Filtrar por categoría | 18. Ordenar por popularidad
19. Ver ingredientes de cada producto (tienda) | 20. Calcular máximo de unidades disponibles

Carrito y Checkout (15)

1. Agregar producto al carrito | 2. Eliminar producto del carrito
3. Cambiar cantidad | 4. Ver total del carrito
5. Aplicar redención de bonos (5 bonos = 5% desc) | 6. Ver resumen antes de comprar
7. Seleccionar tipo de pedido (local/llevar/domicilio) | 8. Ingresar dirección de entrega
9. Ver estimado de entrega (45 min) | 10. Completar compra (crear Pedido + Recibo)
11. Validar stock antes de confirmar | 12. Mostrar recibos digitales
13. Limpiar carrito | 14. Guardar pedido como borrador (future)
15. Historial de carritos anteriores (future)

Pedidos (25)

1. Crear pedido (cliente) | 2. Ver lista de pedidos (admin)
3. Ver mis pedidos (cliente) | 4. Cambiar estado (admin)
5. Calcular fecha de entrega estimada | 6. Marcar como entregado + fecha real
7. Marcar como cancelado | 8. Ver detalles del pedido
9. Imprimir pedido | 10. Generar etiqueta de cocina
11. Filtrar por estado | 12. Filtrar por rango de fechas
13. Filtrar por cliente | 14. Ver tiempo transcurrido vs estimado (semáforo)
15. Ver productos con personalizaciones ("Sin cebolla") | 16. Cambiar dirección (si aún pendiente)
17. Cancelar pedido (restricciones) | 18. Reagendar entrega
19. Notificación cuando listo | 20. Enviar factura por email
21. Descargar pedido PDF | 22. Ver costo total + IVA
23. Editar cantidad de ítems (restricciones) | 24. Aplicar descuento manual (admin)
25. Ver historial de cambios de estado

Facturación (10)

1. Crear recibo manualmente | 2. Editar recibo
3. Eliminar recibo | 4. Ver lista de recibos
5. Calcular IVA automáticamente | 6. Asignar bonos en recibo

- 7. Generar factura PDF | 8. Validar que pedido no tenga recibo duplicado
- 9. Exportar reporte de recibos | 10. Auditar cambios en facturas

Dashboard y Analytics (8)

- 1. Ver KPI: Total ventas del período | 2. Ver KPI: Cantidad de pedidos
- 3. Ver KPI: Usuarios activos | 4. Ver gráfico de ventas por día
- 5. Ver pedidos por estado | 6. Ver productos más vendidos
- 7. Ver métodos de pago frecuencia | 8. Filtrar por rango de fechas

Sistema de Bonos (5)

- 1. Ganar bono por compra $\geq$ \$50,000 | 2. Acumular máximo 10 bonos
- 3. Redimir 5 bonos por 5% descuento | 4. Ver saldo de bonos
- 5. Ver historial de bonos ganados/gastados

Seguridad y Autenticación (4)

- 1. Validar credenciales en login | 2. Prevenir fuerza bruta (future)
- 3. CSRF protection | 4. Prevenir open redirect en login

TOTAL: 85+ historias bien definidas derivables de este inventario.

■ CONCLUSIÓN

Este inventario técnico exhaustivo cubre:

- 12 aplicaciones Django integradas
- 25+ modelos de datos con relaciones complejas
- 80+ endpoints funcionales
- 2 signals críticos para automatización (pedido_pre_save, procesar_inventario_por_pedido)
- 4 decoradores de autenticación/autorización
- Sistema de bonos de fidelidad con redención
- Gestión de lotes por insumo con trazabilidad
- Auditoría completa de movimientos con historiales
- Dashboard con KPIs y gráficos interactivos
- Transacciones atómicas con lock pessimista
- Validaciones exhaustivas (regex, Decimal parsing, relaciones)
- Semáforo inteligente de entregas (verde/amarillo/rojo)
- API hora con fallback
- Sistema de carrito en sesión Django
- Compatibilidad con contraseñas legacy + migración automática

Este inventario proporciona suficiente contexto técnico y funcional para:

- Redactar 85+ historias de usuario específicas y medibles
- Identificar todas las validaciones y reglas de negocio
- Entender flujos críticos (checkout, inventario, entregas)
- Planificar sprints con precisión arquitectónica
- Onboarding de nuevos desarrolladores al proyecto
- QA exhaustivo cubriendo 100% de funcionalidades

Documento generado: 20 de April de 2026